

GestRes Pro

Systeme de gestion de ressources d'entreprise

Candidat	CAMARA Ibrahim
N° candidat	2545812845
Formation	Digital School of Paris – IEF2I
Option	BTS SIO – SLAM (Solutions Logicielles et Applications Métiers)
Session	2026
Épreuve	E6 – Présentation orale du projet
Durée de présentation	~25 minutes

Plan de la présentation

#	Section	Durée
01	Introduction & contexte	2 min
02	Analyse du besoin et spécifications	3 min
03	Architecture technique	4 min
04	Fonctionnalités principales	5 min
05	Sécurité et conformité RGPD	3 min
06	Tests et qualité logicielle	3 min
07	Difficultés et solutions	3 min
08	Bilan & compétences BTS SIO	2 min

01 Introduction & Contexte du projet

≈ 2 min

Bonjour à tous,

Je m'appelle **CAMARA Ibrahim**, je suis étudiant en deuxième année de BTS SIO option SLAM à la Digital School of Paris. Je vais vous présenter aujourd'hui mon projet de session 2026 : **GestRes Pro**, une application de gestion de ressources d'entreprise.

Contexte et origine du projet

La problématique de départ est simple : dans de nombreuses entreprises, la gestion des ressources — qu'il s'agisse d'équipements informatiques, de véhicules, de badges d'accès ou de comptes applicatifs — est encore effectuée manuellement, via des fichiers Excel dispersés, sans traçabilité fiable ni conformité RGPD. Cela entraîne des pertes de matériel, des oublis d'affectation, des risques de sécurité et des difficultés lors d'audits.

C'est dans ce contexte que j'ai conçu et développé **GestRes Pro** : une application de bureau complète, sécurisée et conforme au RGPD, qui centralise le cycle de vie complet de toutes les ressources d'une organisation.

Durée et périmètre de réalisation

Le projet a été réalisé **seul**, de **septembre 2025 à mai 2026**, dans le cadre de l'épreuve E6 du BTS SIO. Il représente environ **4 971 lignes de code source** réparties sur plus de 30 fichiers Python.

NOTES ORATEUR · ≈ 2 min

- Regarder le jury, sourire, respirer.
- Insister sur le mot 'seul' — projet individuel.
- Mentionner que la démonstration sera faite en live avec la société test CleanPro Services.
- Ne pas rentrer dans les détails techniques ici, juste planter le décor.

Identification des acteurs

Avant d'écrire la première ligne de code, j'ai conduit une analyse des besoins en m'appuyant sur des cas d'usage concrets. Trois types d'utilisateurs ont été identifiés :

Rôle	Responsabilités principales	Accès
Super Administrateur	Crée les espaces entreprise, configure les types de ressources, définit les rôles	Toutes les permissions ["all"]
Administrateur d'entreprise	Gestion complète des ressources et utilisateurs, accès aux logs, génération de contrats	8 permissions spécifiques
Employé	Consultation de ses propres affectations, signature électronique de contrats, historique personnel	4 permissions restreintes

Types de ressources gérées

- **Équipements informatiques** : PC, laptops, serveurs, périphériques
- **Comptes applicatifs** : Accès aux logiciels, licences
- **Véhicules** : Flotte automobile de l'entreprise
- **Badges / cartes d'accès** : Contrôle d'accès physique aux locaux
- **Téléphonie** : Mobiles, fixes, abonnements
- **Mobilier & équipements** : Bureau, chaises, équipements divers
- **Ressources personnalisées** : Types définis librement par l'admin

Contraintes identifiées

- Conformité RGPD / CNIL : minimisation des données, droit à l'oubli, journaux d'accès
- Sécurité : aucune requête SQL brute, mots de passe hachés, intégrité des contrats
- Portabilité : application bureau autonome, base de données locale SQLite
- Traçabilité : chaque action sur une ressource est enregistrée dans un journal d'audit
- Multi-entreprise : chaque société dispose de son espace de données isolé

NOTES ORATEUR · ≈ 3 min

- Mentionner que le cahier des charges a orienté toutes les décisions d'architecture.
- Insister sur le fait que le RGPD n'est pas une contrainte ajoutée après coup : c'est pensé dès le départ.
- Si le jury pose une question sur les exigences, renvoyer aux compétences BTS SIO (A1.1, A1.2, A2.1).

Stack technologique

Couche	Technologie	Rôle
Langage	Python 3.14	Langage principal
Interface graphique	PySide6 ≥ 6.6	Framework Qt — vues et widgets
ORM / Base de données	SQLAlchemy 2.0 + SQLite 3	Modèles, requêtes, migrations
Hachage	bcrypt 4.1 (rounds=12)	Sécurisation des mots de passe
PDF	ReportLab 4.0 + PyMuPDF 1.27	Génération et remplissage de contrats
Tests	pytest 155 tests	Tests unitaires et d'intégration
Backup	Module utils/backup.py	Sauvegarde locale automatique de la BDD

Pattern MVC — séparation des responsabilités

J'ai choisi le pattern **MVC (Modèle-Vue-Contrôleur)** pour garantir une séparation claire des responsabilités et faciliter la maintenance. Voici comment s'organise le code :

MODÈLES (src/models/) — 10 modèles SQLAlchemy

- user.py — utilisateurs, rôles, hachage bcrypt
- company.py — espaces entreprise multi-tenant
- resource.py + resource_type.py — ressources et leurs catégories
- assignment.py — affectations ressource ↔ utilisateur
- contract.py — contrats PDF signés électroniquement
- audit_log.py — journal de toutes les actions
- base.py — configuration moteur SQLAlchemy

CONTRÔLEURS (src/controllers/) — 6 contrôleurs

- auth_controller.py — authentification, sessions, tokens
- company_controller.py — CRUD entreprises
- resource_controller.py — CRUD ressources et types

- `assignment_controller.py` — gestion des affectations
- `contract_controller.py` — génération, signature, vérification PDF
- `user_controller.py` — gestion des utilisateurs internes

VUES (`src/views/`) — 10 vues PySide6

- `login_view.py` — écran d'authentification
- `main_window.py` — fenêtre principale, navigation par onglets
- `dashboard_view.py` — tableau de bord avec statistiques
- `resources_view.py` / `users_view.py` — gestion CRUD avec filtres
- `assignments_view.py` / `contracts_view.py` — affectations et contrats
- `logs_view.py` — journal d'audit filtrable
- `companies_view.py` — liste des espaces entreprise

UTILITAIRES (`src/utls/`) — 4 modules

- `security.py` — bcrypt, SHA-256, validation SIRET, `sanitize_input()`
- `validators.py` — validation email, SIRET, forces de mot de passe
- `backup.py` — sauvegarde automatique de la BDD SQLite
- `maintenance.py` — purge RGPD, anonymisation, nettoyage

NOTES ORATEUR · ≈ 4 min

- Pointer chaque couche MVC sur un schéma si disponible (diagramme page 5 de l'annexe).
- Expliquer pourquoi MVC : testabilité, maintenabilité, respect du principe de responsabilité unique.
- Si question sur SQLAlchemy : 'j'utilise l'ORM pour ne jamais écrire de SQL brut, ce qui élimine le risque d'injection SQL'.
- Si question sur PySide6 vs Tkinter : 'PySide6 offre des widgets professionnels, des threads Qt et une meilleure expérience utilisateur'.

04 Fonctionnalités principales

≈ 5 min

Je vais maintenant vous présenter les fonctionnalités clés de GestRes Pro, que je vais illustrer avec la société de démonstration **CleanPro Services** — une société de nettoyage fictive préconfigurée dans la base de données.

a) Authentification sécurisée

L'écran de connexion valide les identifiants via bcrypt (rounds=12). Un token de session est généré avec `secrets.token_hex(32)`. Trois tentatives échouées verrouillent le compte.

b) Tableau de bord

À la connexion, l'administrateur accède à un tableau de bord affichant en temps réel : le nombre de ressources actives, les affectations en cours, les contrats en attente de signature, et les dernières actions du journal d'audit.

c) Gestion des ressources

Création, modification, suppression et archivage de ressources. Chaque ressource est typée (matériel, véhicule, compte...), possède un état (disponible, affecté, en maintenance, hors service) et un numéro de série unique. La vue propose des filtres avancés par type, état, et recherche textuelle.

d) Gestion des affectations

Un employé peut se voir affecter une ou plusieurs ressources. L'affectation enregistre la date de début, la date de fin prévue, et génère automatiquement un contrat PDF. Chaque retour de ressource est tracé dans le journal d'audit.

e) Génération et signature de contrats PDF

À chaque affectation, un contrat PDF est généré avec ReportLab. L'employé peut le signer électroniquement depuis l'application. La signature est calculée en SHA-256 : `hash_contenu + données_signataire + horodatage ISO`. La vérification d'intégrité est possible à tout moment.

f) Journal d'audit complet

Chaque action (connexion, création, modification, suppression, signature) est enregistrée avec : utilisateur, action, table cible, ancienne valeur, nouvelle valeur, timestamp. La rétention est de 5 ans (`AUDIT_LOG_RETENTION_DAYS=1825`) conformément au RGPD.

g) Gestion multi-entreprise

Le super administrateur peut créer plusieurs espaces entreprise (validation du SIRET par algorithme de Luhn). Chaque espace est totalement isolé. Les administrateurs n'ont accès qu'aux données de leur propre société.

h) Maintenance et conformité RGPD

Un module de maintenance permet la purge automatique des données expirées, l'anonymisation pour le droit à l'oubli, et la sauvegarde locale de la base de données. La rétention par défaut des données personnelles est de 3 ans (DEFAULT_RETENTION_DAYS=1095).

Données de démonstration — CleanPro Services

Élément	Détail
Société	CleanPro Services — SIRET 73282932000074
Adresse	12 rue des Lilas, Paris 13e
Administrateur	Marie Dupont (responsable@cleanpro.test / Responsable1!)
Employés	4 employés avec affectations et contrats signés
Ressources	20 ressources (7 types différents)
Affectations	11 affectations actives
Contrats	11 contrats PDF signés électroniquement

NOTES ORATEUR · ≈ 5 min

- Faire la démonstration live pendant cette section (ouvrir l'appli).
- Montrer dans l'ordre : connexion admin → dashboard → liste ressources → une affectation → un contrat PDF → le journal d'audit.
- Si une fonctionnalité ne fonctionne pas en démo, rester calme et décrire le comportement attendu.
- Identifiants demo : responsable@cleanpro.test / Responsable1!
- Super admin : superadmin@gestres.test / SuperAdmin1!

La sécurité et la conformité réglementaire ont été des priorités absolues dans la conception de GestRes Pro, pas des fonctionnalités ajoutées en dernier lieu.

Mesures de sécurité techniques

→ Hachage bcrypt rounds=12

Chaque mot de passe est haché avec bcrypt au facteur de coût 12. Même en cas de fuite de la base de données, les mots de passe restent protégés.

→ Zéro SQL brut

Toutes les requêtes passent par l'ORM SQLAlchemy. Il est structurellement impossible d'effectuer une injection SQL.

→ Sanitisation des entrées

La fonction `sanitize_input()` nettoie tous les champs texte. La validation de force de mot de passe vérifie majuscules, chiffres et caractères spéciaux.

→ Intégrité des contrats SHA-256

Chaque contrat signé produit une empreinte SHA-256 : `hash_contenu + données_signataire + horodatage ISO`. La falsification d'un contrat est détectable immédiatement.

→ Tokens de session sécurisés

`secrets.token_hex(32)` génère 32 octets d'entropie cryptographique par session. Aucun token prédictible.

→ Validation SIRET (Luhn)

L'algorithme de Luhn vérifie la validité mathématique du numéro SIRET, avec un garde explicite contre les SIRET composés de zéros.

Conformité RGPD / CNIL

- Minimisation des données : seules les informations nécessaires sont collectées.
- Rétention configurable : 3 ans pour les données personnelles (`DEFAULT_RETENTION_DAYS=1095`), 5 ans pour les journaux d'audit (`AUDIT_LOG_RETENTION_DAYS=1825`).

- Droit à l'oubli : anonymisation des données de l'utilisateur via le module maintenance.py (remplacement du nom et de l'email par des valeurs anonymisées).
- Journal d'accès : chaque consultation et modification de données personnelles est tracée.
- Stockage local uniquement : aucune donnée n'est transmise vers des serveurs tiers.
- Intégrité des contrats : la signature SHA-256 garantit la non-répudiation.

NOTES ORATEUR · ≈ 3 min

- Si le jury demande pourquoi bcrypt et pas SHA-256 pour les mots de passe : 'bcrypt est volontairement lent et intègre un sel automatique, contrairement à SHA-256 qui est rapide et sans sel'.
- Si question sur le RGPD : mentionner l'article 5 (principes) et l'article 17 (droit à l'effacement).
- Montrer le code de security.py si possible pour prouver la rigueur.

La qualité du code est assurée par une suite de **155 tests pytest** répartis sur 3 fichiers de tests, couvrant les modèles, les contrôleurs et les utilitaires de sécurité.

Organisation des tests

Fichier	Contenu	Portée
test_models.py	Tests des 10 modèles SQLAlchemy	Création, contraintes, relations
test_controllers.py	Tests des 6 contrôleurs	CRUD, authentification, permissions
test_security.py	Tests des utilitaires de sécurité	Hachage, validation, sanitisation
conftest.py	Fixture SQLite in-memory	Isolation totale — database.db jamais touché

Stratégie d'isolation — conftest.py

Le fichier conftest.py met en place une fixture autouse qui redirige tous les accès à la base de données vers une instance SQLite **in-memory**. Cela signifie que les 155 tests s'exécutent sans jamais toucher database.db, garantissant une reproductibilité et une isolation parfaites. Le moteur est patché dans trois modules simultanément : base_module.engine, tests.test_controllers.engine, et tests.test_models.engine.

Exemple de commande d'exécution

```
python -m pytest tests/ -v
```

```
python -m pytest tests/test_security.py -v
```

Bug découvert et corrigé grâce aux tests

Un test a mis en évidence un bug dans la validation SIRET : le numéro '00000000000000' passait la validation de l'algorithme de Luhn car la somme de zéros est un multiple de 10. Un garde explicite a été ajouté dans security.py pour rejeter les SIRET composés uniquement de zéros.

NOTES ORATEUR · ≈ 3 min

- Lancer les tests en live si le temps le permet : `python -m pytest tests/ -v`
- Si le jury demande le taux de couverture : les tests couvrent 100% des chemins critiques (auth, contrats, sécurité).
- Insister sur l'isolation in-memory : 'mes tests ne polluent jamais la base de données de production'.

Tout projet de développement comporte des obstacles techniques. Voici les principaux défis que j'ai rencontrés et comment je les ai résolus.

Difficulté 1 — Gestion des permissions multi-rôles

Problème : implémenter un système de permissions granulaire sans dupliquer la logique dans chaque vue.

Solution : centralisation dans config.py avec un dictionnaire de rôles → liste de permissions. Chaque contrôleur vérifie les permissions via une méthode utilitaire unique, réduisant le code dupliqué.

Difficulté 2 — Isolation des tests

Problème : les premiers tests modifiaient la base de données réelle, rendant les suites de tests non reproductibles.

Solution : création de conftest.py avec une fixture autouse qui substitue le moteur SQLAlchemy par une instance SQLite in-memory, patchée dans tous les modules concernés.

Difficulté 3 — Génération de contrats PDF

Problème : remplir un formulaire PDF officiel tout en insérant des pages personnalisées, avec gestion des polices et des images dont les noms contiennent des apostrophes.

Solution : utilisation de PyMuPDF pour remplir le formulaire existant (wipe + réécriture précise des zones), et de ReportLab pour générer les pages complémentaires. Les images avec apostrophe sont chargées via stream=bytes et non via filename.

Difficulté 4 — Intégrité des signatures de contrats

Problème : garantir qu'un contrat signé ne peut pas être modifié sans que cela soit détecté.

Solution : calcul d'une empreinte SHA-256 combinant le hash du contenu, les données du signataire et un horodatage ISO. La vérification compare l'empreinte stockée avec un recalcul à la demande.

Difficulté 5 — Validation SIRET robuste

Problème : l'algorithme de Luhn acceptait '00000000000000' comme SIRET valide.

Solution : ajout d'un garde explicite avant le calcul de Luhn pour rejeter tout SIRET ne contenant que des zéros. Ce bug a été détecté grâce aux tests unitaires.

NOTES ORATEUR · ≈ 3 min

- Raconter chaque difficulté comme une histoire : 'j'ai d'abord essayé X, ça n'a pas marché parce que Y, alors j'ai fait Z'.
- Les difficultés montrent la maturité technique — ne pas les minimiser.
- Si le jury pose des questions techniques précises sur une solution, répondre sereinement : toutes les solutions sont dans le code.

Ce que j'ai appris et développé

Ce projet m'a permis de développer des compétences transversales couvrant l'intégralité du référentiel BTS SIO SLAM :

→ Conception et modélisation

Analyse du besoin, diagrammes UML (cas d'utilisation, classes, séquences), modélisation de la base de données avec 10 tables et contraintes de clés étrangères.

→ Développement et architecture

Application du pattern MVC sur un projet de ~5 000 lignes, utilisation d'un ORM professionnel (SQLAlchemy 2.0), développement d'interfaces graphiques Qt (PySide6).

→ Sécurité applicative

Hachage bcrypt, tokens sécurisés, validation stricte des entrées, élimination de toute injection SQL, intégrité SHA-256 des documents.

→ Qualité et tests

155 tests automatisés pytest, isolation via fixtures in-memory, détection et correction de bugs réels grâce aux tests.

→ Conformité réglementaire

Intégration native du RGPD : minimisation, rétention, anonymisation, journaux d'accès.

→ Documentation technique

Rédaction du dossier E6, du tableau de synthèse E5, et des annexes officielles.

Perspectives d'évolution

- Version web avec FastAPI + React pour un accès multi-poste
 - Notifications par e-mail lors des échéances de ressources
 - Tableau de bord analytique avec graphiques (matplotlib / Chart.js)
 - Export des journaux d'audit en format CSV / Excel
 - Intégration d'une API SIRET officielle pour la vérification automatique
-

En conclusion, GestRes Pro est une application professionnelle, sécurisée et documentée qui répond à un besoin réel. Ce projet m'a permis de mettre en pratique l'ensemble des compétences du BTS SIO SLAM dans un contexte exigeant. Je suis maintenant disponible pour répondre à vos questions.

NOTES ORATEUR · ≈ 2 min

- Garder la conclusion courte et directe.
- Sourire, regarder le jury, poser les mains à plat sur la table.
- Phrase clé finale : 'GestRes Pro est un projet dont je suis fier, car il est le reflet de ce que j'ai appris pendant ces deux années.'
- Attendre les questions sans montrer de stress — toutes les réponses sont dans le code.

Questions & Réponses

Questions fréquentes et réponses préparées

Q : Pourquoi SQLite et pas PostgreSQL ou MySQL ?

R : SQLite est parfait pour une application de bureau mono-poste : zéro configuration, fichier unique, transactions ACID, suffisant pour des milliers de ressources. SQLAlchemy permet de migrer vers PostgreSQL sans changer le code applicatif si l'application évolue vers une version multi-poste.

Q : Comment garantisiez-vous la conformité RGPD en pratique ?

R : Trois mécanismes concrets : (1) module maintenance.py avec anonymisation et purge automatique selon les délais légaux, (2) journal d'audit tracant chaque accès aux données personnelles, (3) stockage 100% local sans transfert vers des tiers.

Q : Pourquoi PySide6 et pas une appli web ?

R : Le cahier des charges demandait une application de bureau autonome, utilisable sans réseau. PySide6 offre une expérience native, des performances supérieures pour les opérations sur la base de données, et une distribution simplifiée.

Q : Comment fonctionne la signature électronique ?

R : Ce n'est pas une signature cryptographique au sens légal (eIDAS), mais une empreinte d'intégrité SHA-256 qui garantit que le contrat n'a pas été modifié après signature. Elle combine le hash du contenu, l'identité du signataire, et un horodatage ISO non modifiable.

Q : Avez-vous pensé aux cas d'erreur ?

R : Oui — chaque contrôleur gère les exceptions SQLAlchemy, les vues affichent des messages d'erreur explicites, et les 155 tests couvrent les cas limites et les erreurs intentionnelles (mauvais mot de passe, SIRET invalide, etc.).